Structural Testing in Autonomous Driving

A Case Study in COEN448/6761

March 2026

Contents

1 Introduction

Autonomous Driving Systems (ADS) require rigorous verification. While high-level simulation (e.g., CARLA) is standard, unit-level integrity is ensured through **Control Flow Testing (CFT)** and **Data Flow Testing (DFT)**. This document maps theoretical criteria to practical ADS implementations.

2 Control Flow Testing (CFT): Emergency Braking

CFT ensures every logical branch in the decision logic is executed.

2.1 Code Illustration: Obstacle Detection Logic

Listing 1: Emergency Braking Logic (C++)

```
void ProcessBraking(float distance, bool v2x_warn) {
    if (distance < 5.0) {                // Branch A
        TriggerBrake(1.0);               // Full Force
    } else if (v2x_warn) {               // Branch B
        TriggerBrake(0.5);               // Partial Force
    } else {                             // Branch C
        MaintainSpeed();
    }
}
```

2.2 Mapping to Decision Coverage

To achieve 100% Decision Coverage, the test suite must exercise:

- **Test 1:** `distance = 2.0` (Triggers Branch A).
- **Test 2:** `distance = 10.0, v2x_warn = true` (Triggers Branch B).
- **Test 3:** `distance = 10.0, v2x_warn = false` (Triggers Branch C).

3 The Laski-Korel Data Flow Criteria

3.1 Laski-Korel Context: The DU-Path Association

A core contribution of Laski and Korel is the concept of a **Data Flow Context**. For a specific statement, the context is the set of definitions that could potentially reach that statement.

$$Context(S_i) = \{(v, d) \mid d \text{ is a definition of } v \text{ that reaches } S_i\} \quad (1)$$

By testing the context, developers ensure that at any given point in the program, the data being handled is precisely what the "logic" (control flow) expects. The Laski-Korel criteria provide a formal framework for selecting test paths based on the lifecycle of variables. Unlike Control Flow Testing, which focuses on the program's structural branches, Laski-Korel focuses on the **data dependencies** created when a variable is assigned a value (defined) and subsequently read (used).

3.2 Formal Definitions and DU-Chains

Data flow analysis is predicated on three fundamental concepts:

- **Definition (def):** A statement where a value is stored in a variable (e.g., $x = 5$).
- **Use (use):** A statement where the variable's value is accessed.
  - **c-use:** A computational use (e.g., $y = x + 1$).
  - **p-use:** A predicate use in a decision (e.g., $if(x > 0)$).
- **Definition-Clear Path:** A path between a definition and a use where the variable is not redefined by an intermediate statement.

3.2.1 Code Illustration: Sensor Fusion Pipeline

Listing 2: Sensor Fusion Data Flow

```
1 public class SensorFusion {
2     public String calculateAction(double lidar, double radar)
          {
```

```
3           double min_dist;                    // Line 3: def(
                min_dist)
4           if (lidar < radar) {                // Line 4: use(lidar,
                 radar)
5               min_dist = lidar;               // Line 5: def(
                    min_dist)
6           } else {
7               min_dist = radar;               // Line 6: def(
                    min_dist)
8           }
9           if (min_dist < 2.0) return "STOP"; // Line 9: use(
                min_dist)
10          return "PROCEED";
11      }
12  }
```

The Laski-Korel strategy defines several criteria for variable integrity as shown in Table 3.2.1

Criterion	**DU-Pairs**	**ADS Verification Objective**
All-Defs	(5, 9) or (6, 9)	Ensure every sensor source (Lidar or Radar) is used in at least one decision.
All-Uses	(5, 9) **and** (6, 9)	Ensure the "STOP" logic is verified for *both* sensor inputs.
All-DU-Paths	All unique paths	Test the logic under every permutation of sensor priority and outcome.

Table 1: An Illustrating Dataflow Testing Scenario

3.3 Mapping Laski-Korel to Coverage Techniques

Laski and Korel proposed several strategies to ensure that the relationship between data production and consumption is thoroughly exercised. These map directly to standard Data Flow Testing (DFT) techniques:

3.3.1 All-Definitions (All-Defs) Criterion

The **All-Definitions** criterion is the most fundamental level of data flow adequacy. It requires that for every definition of a variable v at statement s_i, there exists at least one test case that follows a definition-clear path to at least one use (either computational or predicate) of v.

- **Formal Mapping:** This criterion ensures the functional reachability of data-originating instructions. By requiring a definition-clear path to a subsequent use, it detects the presence of **"Terminating Definitions"** where an assignment is overwritten or discarded before it can influence the program's observable state or control flow. Effectively, this serves as a structural proof that every definition possesses at least one execution context in which it is not redundant.

3.3.2 Anti-Example: Data Flow Redundancy in Sensor Logic

Consider the following implementation of an autonomous braking margin. This example demonstrates a violation of the All-Definitions criterion through a "Definition-Definition" (D-D) anomaly.

Listing 3: Violation of All-Defs via Data Overwrite

```
public void calculateSafetyMargin(double rawSensorData) {
double margin = 0.5; // Def 1: Initial safety margin

if (v2xEnabled()) {
    margin = getV2XMargin(); // Def 2: Potential redefinition
} else {
    // LOGIC ERROR: The developer hardcodes a reset
    margin = 0.0; // Def 3: This redefinition "kills" Def 1
}

// Use of 'margin'
applyBrake(rawSensorData + margin); // Use at statement s_n
}
```

In the code above, the **All-Definitions** criterion reveals a critical structural flaw that standard Control Flow Testing (e.g., Statement Coverage) might overlook:

1. **The Violation:** There is no executable path where **Def 1** reaches the use at statement s_n. In the `else` branch, **Def 3** overwrites **Def 1** before it can be consumed.

2. **The Data Flow Anomaly:** This represents a "Dead Definition." Even if the code on line 2 is executed, its effect on the system is nullified.

3. **Verification Outcome:** Under Laski-Korel's All-Defs, a test suite would be flagged as *inadequate* because it cannot demonstrate a path where the initial `0.5` margin actually influences the `applyBrake` command. This forces the developer to recognize that the hardcoded reset in the `else` block rendered the previous safety logic obsolete.

3.3.3 All-Uses Criterion

The **All-Uses** criterion represents a significant escalation in testing rigor beyond the All-Definitions requirement. It mandates that for every definition of a variable v at statement s_i, the test suite must exercise at least one definition-clear path to **every** possible computational use (c-use) and predicate use (p-use) that can be reached from s_i.

- **Formal Mapping:** This criterion corresponds to the comprehensive coverage of all valid **Definition-Use (DU) Pairs**. Within the context of Autonomous Driving Systems (ADS), it ensures that a specific data state such as a processed sensor value, correctly propagates to and influences every downstream logic branch and actuator command that relies on that specific definition. While All-Definitions only requires that a value is used *somewhere*, All-Uses ensures it is validated *everywhere* it is relevant.

3.3.4 Anti-Example: Partial Logic Validation in ADS

Consider a module responsible for adjusting vehicle velocity based on a single sensor reading. An anti-example of the All-Uses criterion occurs when a test suite verifies the impact of a variable on one subsystem but fails to verify its impact on another critical decision point.

Listing 4: Violation of All-Uses via Incomplete Path Coverage

```
1 public void velocityControl(double targetSpeed) {
2 double cmdSpeed = targetSpeed; // Def: cmdSpeed
3
4 //skipped code
5 // Use 1 (Predicate): Speed Limit Guard
6 if (cmdSpeed > 120.0) {
```

```
    cmdSpeed = 120.0;
}

// Use 2 (Computation): Engine Torque Calculation
double torque = cmdSpeed * TORQUE_CONSTANT;

// Use 3 (Predicate): Dashboard Warning Light
if (cmdSpeed > 100.0) {
    displayHighSpeedWarning();
}

applyTorque(torque);

//```

}
```

A test suite that satisfies the **All-Definitions** criterion might only provide a single test case where `targetSpeed = 110.0`.

1. **The Testing Gap:** A test case of `110.0` exercises the path from the Definition to **Use 2** and **Use 3**. However, it does not trigger the path where `targetSpeed` exceeds the 120.0 threshold at **Use 1**.

2. **The Violation:** Under the **All-Uses** criterion, this test suite is *inadequate*. To satisfy the criterion, the tester must provide additional cases (e.g., `targetSpeed = 130.0`) to ensure the definition reaches the guard logic at **Use 1**.

3. **Significance:** In safety-critical ADS, failing to satisfy All-Uses means a developer might confirm that a speed setting correctly updates the dashboard (**Use 3**) while remaining unaware that the same speed setting fails to trigger the safety governor (**Use 1**) due to data corruption or a logic error.

3.4 All-DU-Paths Criterion (Laski-Korel Strategy)

The **All-DU-Paths** criterion is the most exhaustive strategy within the Laski-Korel hierarchy. It requires that for every definition of a variable v at statement s_i and every reachable use of v at s_j, the test suite must exercise **every simple path** from s_i to s_j. A simple path is defined as a path where

all nodes are distinct, except possibly the first and last (allowing for exactly one iteration of a cycle).

- **Formal Mapping:** In the context of Autonomous Driving Systems (ADS), this criterion identifies "state-drift" anomalies. While the All-Uses criterion is satisfied by a single path to each use, All-DU-Paths recognizes that a variable's value can be corrupted by specific **inter-iterative dependencies**. It ensures that the data state remains consistent regardless of whether a use is reached directly, after one loop iteration, or through multiple re-entries of a cyclic structure.

3.4.1 Anti-Example: Iterative Data Corruption in Radar Tracking

The following example illustrates a violation of the All-DU-Paths criterion in a tracking module where a definition reaches a use via different loop traversals.

Listing 5: Corrected: All-DU-Paths Violation in Loop Logic

```
1  public void trackObstacle(double initialDist) {
2  double distance = initialDist; // Def: distance (Line 2)
3  while (sensor.isActive()) {    // Loop Entry
4      // Use 1 (Computation): Update distance based on delta
5      distance = distance - sensor.getDelta();
6
7      if (distance < 0) {
8          // LOGIC ERROR: Resetting distance kills previous
               Defs
9          distance = 0.0;         // Def: distance (Line 9)
10     }
11
12     // Use 2 (Predicate): Safety check
13     if (distance < SAFE_THRESHOLD) {
14         triggerWarning();       // Use of distance (Line 13)
15     }
16 }
17 }
```

A test suite satisfying **All-Uses** might only execute the loop once.

1. **The Testing Gap:** A single iteration exercises the path from the definition at Line 2 to the use at Line 13. However, it fails to exercise

the **recursive path** where the definition at Line 9 (the reset) reaches the use at Line 13 in the *next* iteration.

2. **The Violation:** The **All-DU-Paths** criterion requires testing the path where the loop repeats. This exposes a "precision loss" bug: if the sensor reports a negative delta, the tracker resets to a hardcoded 0.0, losing the actual physical velocity context for all subsequent iterations.

3. **Significance:** In ADS, testing only one iteration might show the system is safe. However, All-DU-Paths forces the tester to observe the variable across multiple loop traversals, revealing that the tracker "stalls" at zero during sustained signal noise rather than recovering correctly.

4 Weyuker's Seventh Axiom: Antidecomposition

The **Antidecomposition Criterion** is one of the eleven properties proposed by Laski, Korel, and Weyuker (LKW) to evaluate test data adequacy. It asserts that testing a system in its entirety does not guarantee that its individual components have been tested with sufficient rigor.

> *Axiom Statement:* There exist programs P and Q, where Q is a component of P, such that a test set T is adequate for P, but the restriction of T to Q is not adequate for Q.

In the context of Autonomous Driving Systems (ADS), this property highlights a critical risk: high-level integration tests may exercise all logical branches of a decision module without sufficiently stressing the underlying sensor drivers or data acquisition layers.

4.1 Illustrative Example: Vehicle Speed Verification

Consider an ADS module P (`VehicleSpeedChecker`) that utilizes a subcomponent Q (`SpeedSensor`) to determine if a vehicle is exceeding local speed limits.

Listing 6: Implementation of Program P and Component Q

```
class SpeedSensor {// Component Q: Interfaces with hardware
   to fetch velocity

public Integer getSpeed() {// Simulates intermittent hardware
    failure
  return (Math.random() > 0.1) ? (int) (Math.random() * 120)
     : null;
}
}

class VehicleSpeedChecker {

private SpeedSensor sensor;

public VehicleSpeedChecker(SpeedSensor sensor) {
    this.sensor = sensor;
}

// Program P: High-level decision logic
public boolean isOverspeeding(int speedLimit) {
    Integer speed = sensor.getSpeed();
    if (speed == null) {
        throw new IllegalStateException("Sensor␣failure:␣data
           ␣unavailable");
    }
    return speed > speedLimit;
}
}
```

4.2 Formal Adequacy Analysis

To demonstrate the **Antidecomposition** criterion, we evaluate an Autonomous Driving System (ADS) module P (`VehicleSpeedChecker`) that utilizes a sub-component Q (`SpeedSensor`).

Fault Category	Detected by T (System P)	Detected by TQ (Component Q)	Rationale
Logic Errors	Yes	No	T verifies if the "Overspeeding" flag flips correctly based on the input limit.
Negative Velocity	No	Yes	P treats −10 as "not overspeeding"; Q must identify this as a hardware failure.
Out-of-Range Data	No	Yes	P sees 500 km/h as "overspeeding" (technically correct) but misses the underlying sensor fault.
Data Availability	Yes	Yes	Both layers typically handle null or missing data streams.

Figure 1: Compared Adequacy of two test sets T and T_Q

Listing 7: Test Set T: Adequate for P, Inadequate for Q

```
// Test Set T focuses exclusively on high-level logic outcomes

@Test
void testOverspeedingLogic() {
    VehicleSpeedChecker checker = new VehicleSpeedChecker(new SpeedSensor());
    // Case 1: Testing 'True' branch for overspeeding
    // Adequate for P as it confirms the logic handles speed > limit
    assertTrue(checker.isOverspeeding(0));

    // Case 2: Testing 'False' branch for overspeeding
    // Adequate for P as it confirms the logic handles speed < limit
    assertFalse(checker.isOverspeeding(130));
}
```

While the test set T achieves 100% branch coverage for the decision logic in P, it is fundamentally inadequate for Q. To satisfy the Antidecomposition axiom, a dedicated test set T_Q is required to validate the internal constraints of the sensor.

Listing 8: Dedicated Unit Tests for Component Q

```
@Test
void testSensorIntegrity() {SpeedSensor sensor = new SpeedSensor();Integer speed = sensor.getSpeed();if (speed != null) {
    // Ensuring Q adheres to physical constraints independently
    assertTrue(speed >= 0, "Velocity cannot be negative");
    assertTrue(speed <= 250, "Velocity exceeds vehicle physical limits");
}
}
```

4.2.1 Significance for ADS Safety

This analysis confirms that in safety-critical domains like autonomous driving, "Top-Down" testing alone is insufficient. Under the LKW framework, developers must prove adequacy for each component Q individually. This ensures that sensor faults such as non-physical speed values are caught at the source before they are consumed by a high-level logic that might inadvertently treat a "faulty" value as a "valid" input.

5 Conclusion

Consistency in ADS testing is achieved by moving from branch coverage (CFT) to variable lifecycle integrity (DFT). Applying the L-K criteria ensures that sensor data is never "lost" in the logic, while Weyuker's axioms remind us that comprehensive system testing does not replace rigorous unit testing for individual sensors.